Notes

Slide 1 : calcul de requêtes, algorithme de Yannakakis, acyclique, complexité linéaire

Slide 3 : Cadre théorique.

Slide 4 : Relations/Tables, schéma SQL, importance du coût de la jointure.

Slide 5 : Produit cartésien, explosion combinatoire, données intermédiaires inutiles.

Slide 6 : Définition (AND), SQL simple, limite (pas de LIKE complexe/négation).

Slide 7 : Acyclique vs Cyclique, hypergraphe, réduction GYO, complexité simplifiée.

Slide 8 : Coeur du sujet.

Slide 9 : Sommets = Relations, connexité des variables partagées, arbre de jointure.

Slide 10 : *Full Reducer*, 3 phases :

- 1. *Bottom-up* : élimination "tuples pendants" (semi-jointure).
- 2. *Top-down* : propagation globale.
- 3. *Jointure finale* : calcul sans déchet.
- Complexité linéaire (entrée + sortie).

Slide 11 : Implémentation Python, validation pratique.

Slide 12 : Communes/Départements/Régions, données réelles (35k lignes), structure SQL.

Slide 13 : `Instance.query`, classes Python, gain de performance version double parcours.

Slide 14 : Requête Chef-lieu, efficacité confirmée, temps de réponse divisés par deux.

Slide 15 : Benchmark, temps vs taille DB, validation de la linéarité (pente constante).

Slide 16 : SQLite (C) vs Python, même comportement asymptotique, différence de constante.

Slide 17 : Bilan.

Slide 18 : Efficacité théorique validée, limites Python (gestion mémoire), parallélisation possible. Ajout de filtres (WHERE) dans la phase de réduction.

Slide 20/21 : Détails GYO, NP-complétude (3-SAT/Clique), complexité des preuves.